SkyTech Airlines: A Requirements-Driven Java Web Architecture for Airline Reservation and Passenger Management with Role-Based Access and Transactional Booking

Mohammed Sahil
RTU Roll No. 24EARCS095
Department of CSE
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

Mukesh Sah
RTU Roll No. 24EARCS097
Department of CSE
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

Muskan Raj
RTU Roll No. 24EARCS098
Department of CSE
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

Nikhil Ranjan Choubey
RTU Roll No. 24EARCS103
Department of CSE
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

Prashant Chaudhary
RTU Roll No. 24EARCS115
Department of CSE
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

Er. Ram Babu Buri
Project Mentor
Associate Professor
Arya College of Engineering & I.T.
Jaipur, Rajasthan, India

***Abstract*—Airline reservation depends on several interdependent activities: discovering flights, recording passenger information, issuing booking references, producing tickets, handling cancellations and maintaining schedules. When these are handled manually or through poorly coordinated tools, records become inconsistent, booking references are hard to trace and administrators lack a reliable view of operations. This paper presents the design of SkyTech Airlines, a centralized Java web application that addresses these problems within an academic scope. The system recognizes two roles (Passenger and Administrator) and divides the work into five modules: authentication and user management, flight search and availability, booking and passenger management, ticketing and cancellation, and administrative flight management. The architecture combines JavaServer Pages, Jakarta Servlets, JDBC and a MySQL database on Apache Tomcat 10.1, with server-side role-based access control, salted password hashing, parameterized SQL and transactional booking and cancellation. The paper reports the requirements specification (42 functional requirements), architecture, data model and verification plan. It describes planned capabilities only; no implementation or test result is claimed.**

***Index Terms*—airline reservation, Java web application, JSP, Jakarta Servlets, JDBC, MySQL, role-based access control, PNR, transactional booking.**

I. INTRODUCTION AND PROBLEM STATEMENT

Online reservation has become the normal way of booking air travel. A reservation system lets a passenger create an account, search for flights on a given date, choose a flight, enter passenger details and later retrieve or cancel the booking [1]. Early automated systems were built for the airlines themselves and later opened to travel agencies, and today global distribution systems and proprietary platforms handle the business [2]. These systems are complex and costly to operate, which puts them out of reach of small projects and teaching environments. Building a comparable system as a student project nevertheless raises three recurring problems.

First, **inconsistent records**: where reservation work is done by hand, passenger records are duplicated or inconsistent, and ticket and cancellation records disagree. Second, **poor traceability**: flights are slow to search and booking references are hard to trace. Third, **fragmented oversight**: administrators cannot see flights and bookings in one place. A further difficulty is that a reservation application combines authentication, validation, concurrent seat updates and relational design in a single problem, so security and data-integrity rules must be written down before coding begins.

SkyTech Airlines addresses these problems with a single relational database exposed through role-restricted web pages, a unique PNR (booking reference) for every booking, and validation and transactional rules governing every change to a reservation. The work was specified in a Software Requirements Specification (SRS) for the 2026–2027 academic session at Arya College of Engineering & IT, Jaipur, and this paper condenses that specification. Section II states objectives, Section III reviews related work, Section IV defines requirements, Section V describes the architecture, Section VI the methodology, Section VII the verification plan, Section VIII expected outcomes, and Sections IX–XI limitations, future work and conclusions.

The contributions of this paper are:

1. A requirements-driven design for an airline reservation system with five modules and 42 uniquely identified functional requirements.
2. A layered Java web architecture in which servlet filters enforce authentication and role on every protected request.
3. Transactional booking and cancellation rules with a unique PNR, idempotent cancellation and database-enforced integrity.
4. A written verification plan and acceptance matrix, with an explicit statement that no test has yet been executed.

II. PROJECT OBJECTIVES

The objectives of SkyTech Airlines are:

1. **Centralized data:** hold users, flights, bookings, passengers and tickets in one MySQL database.
2. **Modular scope:** divide the functionality into five modules, each owned by one team member.
3. **Secure access:** enforce Passenger and Administrator roles on the server, never by hiding links.
4. **Traceable bookings:** issue a unique PNR for every booking and keep ticket status consistent with booking status.
5. **Atomic changes:** make booking and cancellation all-or-nothing database transactions.
6. **End-to-end workflow:** cover registration, flight search, booking, ticketing, cancellation and administrative flight management.

7. **Honest reporting:** separate targets from results, and claim nothing as implemented or tested until it has been.

III. RELATED WORK AND TECHNICAL BACKGROUND

A. Prediction, Pricing and Overbooking

Abdella et al. survey airline ticket price and demand prediction using historical fares, purchase dates, holidays, weather and seat availability [3]. Dalalah et al. study voluntary overbooking and its effect on customer satisfaction and pricing [4], and Narangajavana et al. examine how airline fares change with season and booking time [5].

B. Reservation System Structure

Wei et al. apply model checking to a BPEL-based airline ticket reservation system [1]. Lu analyzes airline e-commerce strategies in ticket distribution [2]. Mushtaq and Riaz consider flexible reservation systems built around service quality attributes [6], Sadreddini proposes a cancellation protection service for online reservation [7], and Pinto Lopes et al. study the potential impact of blockchain on the aviation industry [8].

C. Java Web Stack and Security Guidance

The application targets Java 21 [9], JavaServer Pages (Jakarta Pages) [10] and Jakarta Servlet 6.0 [11], the version aligned with Apache Tomcat 10.1 [12]. Data access uses JDBC [13] against MySQL 8.0 [14], built with Apache Maven [15]. Security controls follow the OWASP guidance [16].

D. Positioning

The surveyed works concentrate on prediction, pricing, overbooking, distribution strategy or emerging technology. They give less attention to the specification of a complete, testable reservation application whose security and data-integrity rules are written before coding begins. SkyTech Airlines fills that gap with unique requirement identifiers, an explicit separation of functional and non-functional requirements and testable acceptance conditions. Price and demand prediction, overbooking management and blockchain storage are outside its baseline scope.

IV. SYSTEM REQUIREMENTS AND SCOPE

A. Functional Requirements

Every functional requirement (FR-01 to FR-42) has a unique identifier and uses “shall” for mandatory behavior. It states the actor, preconditions, processing, failure conditions, priority and an acceptance condition. The scope is divided into five modules (Table I). Module 1 supplies identity and role to all others; Module 2 supplies valid flights; Module 3 turns a selected flight and passenger details into a booking; Module 4 manages the booking lifecycle; and Module 5 maintains the flight data on which the others depend.

TABLE I
MODULES AND RESPONSIBILITIES

No.	Module	Key responsibilities
1	Authentication & User Mgmt.	Registration, login/logout, profile, roles, sessions (FR-01–08)
2	Flight Search & Availability	Search, results, availability, selection (FR-09–14)
3	Booking & Passenger Mgmt.	Passenger details, PNR, seat allocation, confirmation (FR-15–23)
4	Ticketing & Cancellation	Ticket, history, PNR lookup, cancellation (FR-24–33)
5	Admin Dashboard & Flights	Flight CRUD, routes, fares, reports (FR-34–42)

B. Non-Functional Requirements

The 41 non-functional requirements (NFR-01 to NFR-41) are phrased so that they can be checked by inspection, test or measurement. Table II summarizes them; performance values are targets, not results.

TABLE II
NON-FUNCTIONAL REQUIREMENTS

Concern	Requirement
Deployment	Java 21; Jakarta Servlet 6.0; Apache Tomcat 10.1; Maven; WAR packaging
Database	MySQL 8.0; foreign keys; UNIQUE and CHECK constraints
Security	Salted adaptive password hashes; prepared statements; output encoding; anti-CSRF token; new session ID at login; HttpOnly cookies
Performance (target)	About 3 s for typical pages; about 2 s for flight search on a stated data size, under a defined load such as 10 concurrent users
Usability	Consistent navigation; marked required fields; explicit confirmation of booking and cancellation
Accessibility	Keyboard operation; labelled controls; sufficient contrast; feedback not conveyed by colour alone
Reliability	Transactional changes; controlled error messages; documented manual backup and restore

C. Roles and Scope

A **Passenger** registers, logs in, manages a profile, searches flights, books, views tickets and history, and cancels their own bookings. An **Administrator**, a provisioned trusted user, manages flights, routes, schedules and fares and reviews bookings and reports. In scope are the five modules and a MySQL database. Out of scope are live airline inventory, payment processing, email delivery, price or demand prediction, overbooking management and blockchain storage. The system depends on no external reservation, payment or identity service, and seat availability reflects its own database only.

V. SYSTEM ARCHITECTURE

A. Overall Structure

SkyTech Airlines is a self-contained, database-driven web application. Passengers and administrators use a standards-compliant browser; a server-side application inside a servlet container processes each request, and a MySQL database stores all persistent data. The system uses a layered design (Fig. 1). JSP pages render forms and results and contain no database code. Servlet filters check authentication and role on every protected request, and servlets call service classes that apply the reservation rules. DAO classes encapsulate JDBC access and manage transaction boundaries. Results return along the same path, and the servlet forwards or redirects to a JSP.

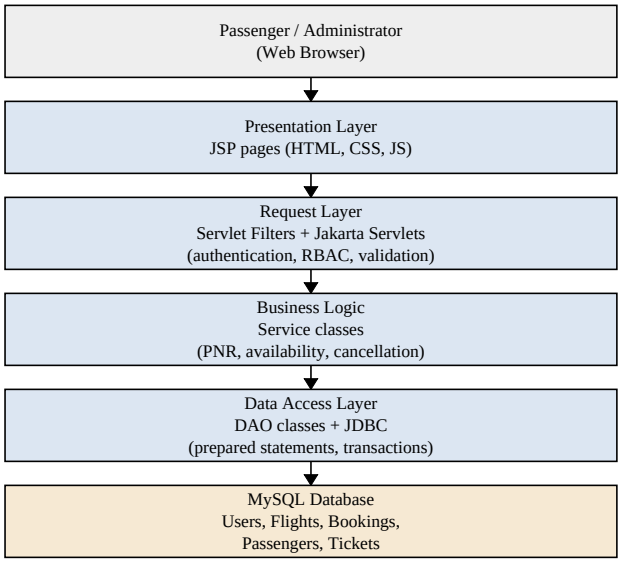

Fig. 1. Proposed layered system architecture of SkyTech Airlines.

B. Use Cases and Access Control

Fig. 2 shows the two actors and their use cases. Login and logout are shared. Every use case is subject to authentication and authorization, and administrative functions are reachable only by the ADMIN role. Authorization is enforced on the server and never relies on hiding links.

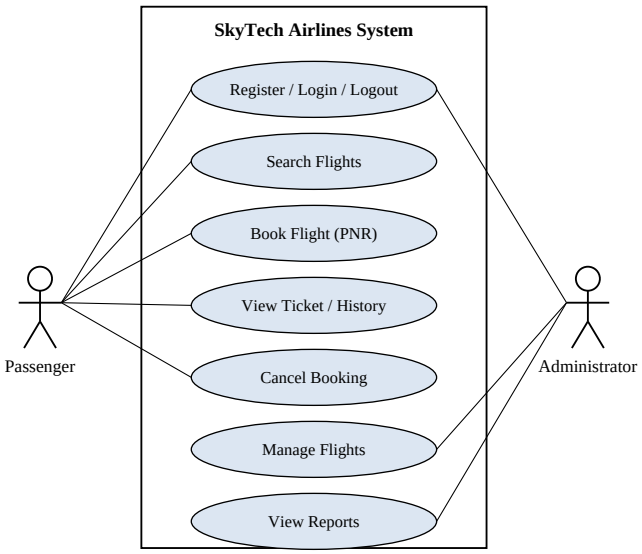

Fig. 2. UML use-case diagram (proposed).

C. Booking Flow

Fig. 3 shows the booking flow. The flight is re-validated on the server after selection, because only the flight identifier is passed on and a tampered or stale identifier must be rejected. Before a booking is committed, the server checks that the flight exists, is ACTIVE, has not departed and has enough seats, and that the user is authenticated. The steps from availability re-check to insertion run inside one transaction that commits on success and rolls back on any failure, so a failure leaves no partial data. A random alphanumeric PNR is generated and protected by a UNIQUE constraint, with regeneration on collision. Payment processing is outside the baseline scope.

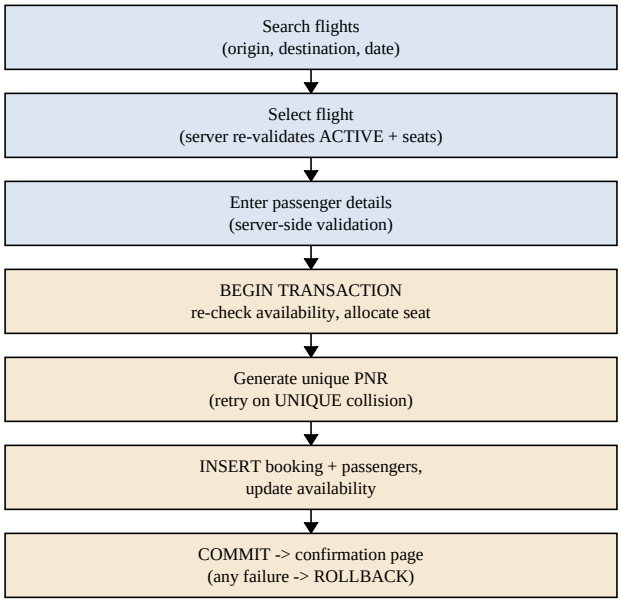

Fig. 3. Flight booking flow with transactional steps.

D. Cancellation Logic

The cancellation logic is shown in Fig. 4. Cancellation checks ownership, status and eligibility, sets the booking and ticket to CANCELLED in one transaction, restores availability, and is conditioned on the current status so that a repeated request changes nothing. Unknown references, bookings owned by another user, bookings already cancelled and ineligible bookings each end with a specific message. Another passenger's PNR receives the same response as an unknown one, so its existence is not revealed.

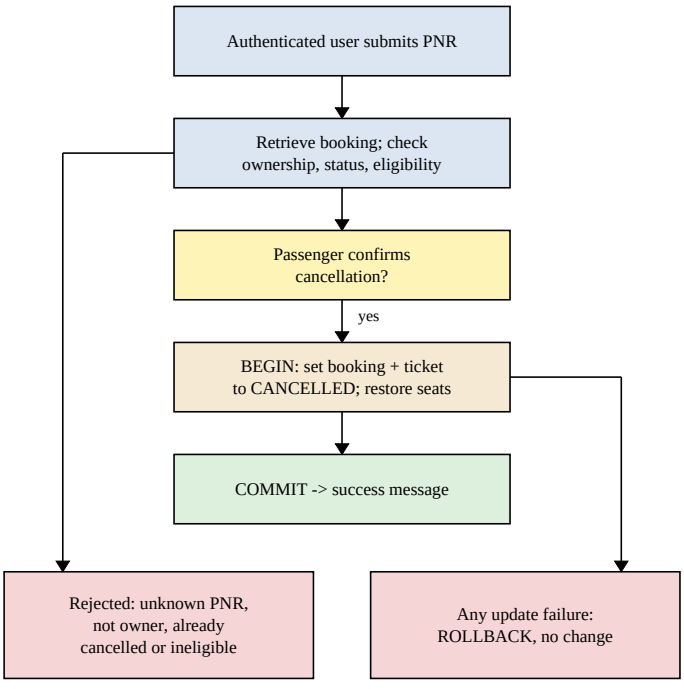

Fig. 4. Ticket cancellation activity diagram (proposed).

E. Data Architecture

The proposed schema has five tables (Fig. 5, Table III). Users hold a unique normalized email, a password hash and a role limited to PASSENGER or ADMIN. Flights carry origin, destination, times, fare, available seats and status (ACTIVE or INACTIVE), with a unique flight number and departure time. A booking belongs to one

user and one flight, stores its fare at booking time so later fare changes do not alter history, and has a status of CONFIRMED or CANCELLED. A booking has one or more passengers and at most one ticket (ISSUED or CANCELLED).

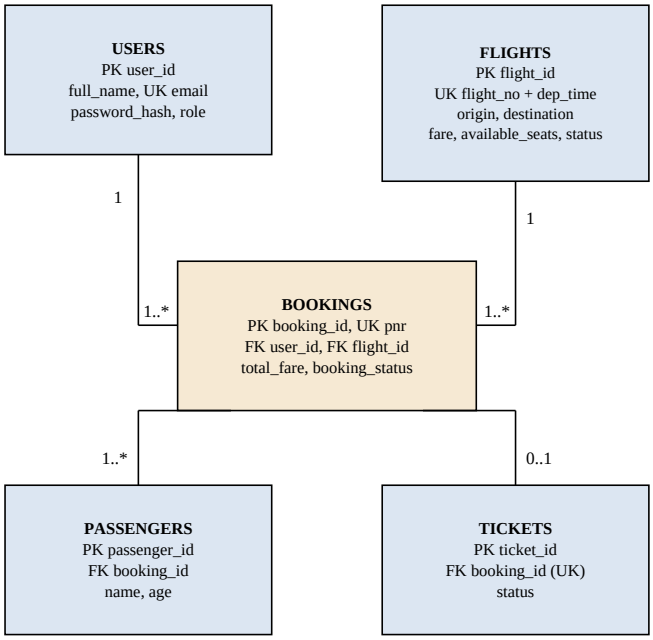

Fig. 5. Proposed entity-relationship diagram.

TABLE III
INTEGRITY RULES ENFORCED BY THE DATABASE

Rule	Mechanism
Record identity	Primary and foreign keys
Booking reference	UNIQUE pnr
Required data	NOT NULL required fields
Valid values	Positive fares; non-negative seats; CHECK on status and role
Consistency	Ticket status must agree with booking status
History	Referenced rows are never hard-deleted; flights are deactivated

F. Security Architecture

Security follows the OWASP guidance [16]. Passwords are stored only as salted adaptive hashes (for example bcrypt, PBKDF2 or Argon2). All SQL uses prepared statements to prevent injection. Dynamic output is encoded against cross-site scripting, state-changing requests carry an anti-CSRF token, and a new session identifier is issued at login with cookies marked HttpOnly. Error pages never expose stack traces, SQL or credentials, and database credentials are kept outside public source control.

VI. METHODOLOGY AND IMPLEMENTATION PLAN

A. Technology Stack

The application is written in Java 21 [9], which offers platform independence, a mature ecosystem and strong typing for a multi-developer project. The presentation layer uses JSP to generate HTML; requests are handled by Jakarta Servlet 6.0 components and filters aligned with Tomcat 10.1; data access uses JDBC against MySQL 8.0. A relational database suits the design because bookings, passengers and tickets depend on each other through foreign keys and need transactional guarantees. The project is built with Maven, packaged as a WAR file, and shared through Git and GitHub.

B. Requirements-Engineering Method

The specification follows a requirements-engineering method: unique identifiers, “shall” statements, an explicit split between functional and non-functional requirements, and acceptance conditions that can be tested. Test cases and a requirements-to-acceptance matrix are derived from the requirements, so every requirement can be traced to at least one check.

C. Module Ownership

Each of the five modules is owned by one team member (Table I). Because Module 1 supplies identity and role, Module 2 supplies valid flights and Module 5 maintains the flight data, integration follows the dependency order of the modules, with regression tests run after each integration.

VII. TESTING AND VERIFICATION PLAN

Verification is layered so that each level catches a different class of fault. **All test cases are currently unexecuted, and outcomes will be recorded only after they are run.**

A. Test Levels

- **Unit tests** of validation, hashing, PNR generation and status transitions.
- **Integration tests** of the module interfaces.
- **System tests** of end-to-end passenger and administrator workflows.
- **Security and usability tests**, and **regression tests** after each integration.

B. Planned Security Cases

TABLE IV
PLANNED SECURITY TEST CASES

#	Case	Expected	Status
1	Passenger requests an administrator URL	Access denied	Unexecuted
2	Tampered flight identifier	Rejected by server	Unexecuted
3	SQL- and script-injection input	Treated as data; output encoded	Unexecuted
4	Another passenger’s PNR	Same reply as unknown PNR	Unexecuted
5	Protected page after logout	Access denied	Unexecuted

C. Acceptance

Acceptance is judged by the requirements-to-acceptance matrix. Non-functional targets such as the 3 s page and 2 s search figures are to be measured under a defined load; they are targets, not results.

VIII. EXPECTED OUTCOMES AND DISCUSSION

A. Expected Benefits

The expected benefits are centralized records, less manual effort, more consistent booking information and easier booking management. They are expectations of the design and have not been measured.

B. Design Decisions

Server-side validation. Only the flight identifier is passed between steps, and the server re-validates it, so a tampered or stale identifier is rejected.

Transactions for booking and cancellation. Availability re-check, seat allocation and insertion form one unit of work, so concurrent or failed requests cannot leave partial records.

Fare stored on the booking. Later fare changes do not alter booking history.

Deactivate, never delete. Flights referenced by bookings are made INACTIVE so that history is preserved.

Non-revealing responses. Another passenger's PNR gets the same reply as an unknown one, so its existence is not disclosed.

C. Threats to Validity

This is a requirements and design study. The diagrams and schema must still be reconciled with the implemented code, no feature is claimed as implemented or tested, and no performance, load or usability measurements exist.

IX. LIMITATIONS

1. Design only: no feature is implemented or tested, and no results are reported.
2. No live airline inventory: seat availability reflects the application's own database.
3. No payment processing or email delivery.
4. Targets only: performance, usability and accessibility figures are to be measured.
5. Reconciliation pending: diagrams and schema must be checked against the eventual code.
6. Baseline scope excludes price or demand prediction, overbooking management and blockchain storage.

X. FUTURE WORK

- Implement and test the five modules and report measured results.
- Add payment-gateway integration and email or SMS notifications.
- Generate PDF tickets and add seat-map selection.
- Add password reset, multi-factor authentication and audit logging.
- Automate backups.
- Connect live inventory or flight-status APIs.
- Layer the price-prediction and overbooking models surveyed in Section III on top of the booking data.

XI. CONCLUSION

This paper presented the design of SkyTech Airlines, a Java web application for airline reservation and passenger management. The specification defines 42 functional requirements in five modules, a set of verifiable non-functional requirements, a layered architecture, a five-table relational schema and a verification plan. Security controls and data-integrity rules are treated as core requirements: server-side role checks, salted password hashing, prepared statements, and transactional booking and cancellation. Because the work is a requirements and design study, implementation and measured results remain future work; stating this plainly keeps the paper's claims honest and gives the team a clear baseline against which to build and test.

REFERENCES

[1] Z. Wei, R. Dong, X. Luo, and F. Liu, "Model checking airline tickets reservation system based on BPEL," in *Proc. 3rd Int. Conf. Genetic and Evolutionary Computing*, IEEE, 2009, pp. 1–6.

[2] W. Lu, "An analysis of airline e-commerce strategies in ticket distribution," in *Proc. Int. Conf. Service Systems and Service Management*, IEEE, 2007, pp. 1–5.

[3] J. A. Abdella, N. Zaki, K. Shuaib, and F. Khan, "Airline ticket price and demand prediction: A survey," *J. King Saud Univ.—Comput. Inf. Sci.*, 2019, doi: 10.1016/j.jksuci.2019.02.001.

[4] D. Dalalah, U. Ojiako, and M. Chipulu, "Voluntary overbooking in commercial airline reservations," *J. Air Transport Manage.*, vol. 86, p. 101835, 2020, doi: 10.1016/j.jairtraman.2020.101835.

[5] Y. Narangajavana, F. J. Garrigos-Simon, J. S. García, and S. Forgas-Coll, "Prices, prices and prices: A study in the airline sector," *Tourism Manage.*, vol. 41, pp. 28–42, 2014, doi: 10.1016/j.tourman.2013.08.008.

[6] A. Mushtaq and S. Riaz, "Flexible airlines reservation systems using service quality attributes of airlines," in *Proc. 6th Int. Conf. Innovative Computing Technology (INTECH)*, IEEE, 2016, pp. 1–5.

[7] Z. Sadreddini, "A novel cancellation protection service in online reservation system," *IEEE Access*, vol. 8, pp. 129094–129107, 2020.

[8] D. Pinto Lopes, P. Rita, and H. Treiblmaier, "The impact of blockchain on the aviation industry: Findings from a qualitative study," *Res. Transp. Bus. Manage.*, 2021, Art. no. 100669, doi: 10.1016/j.rtbm.2021.100669.

[9] Oracle, "Java documentation." [Online]. Available: https://docs.oracle.com/en/java/

[10] Eclipse Foundation, "Jakarta Pages (JSP) specification." [Online]. Available: https://jakarta.ee/specifications/pages/

[11] Eclipse Foundation, "Jakarta Servlet specification." [Online]. Available: https://jakarta.ee/specifications/servlet/

[12] Apache Software Foundation, "Apache Tomcat documentation." [Online]. Available: https://tomcat.apache.org/

[13] Oracle, "Java Database Connectivity (JDBC) documentation." [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

[14] Oracle, "MySQL documentation." [Online]. Available: https://dev.mysql.com/doc/

[15] Apache Software Foundation, "Apache Maven guides." [Online]. Available: https://maven.apache.org/guides/

[16] OWASP Foundation, "OWASP cheat sheet series." [Online]. Available: https://cheatsheetseries.owasp.org/